

**LAPORAN PERTEMUAN
PRAKTIKUM STRUKTUR DATA DAN ALGORITMA
PERTEMUAN 4
IMPLEMENTASI STACK DAN QUEUE BERBASIS LINKED LIST**



Disusun Oleh:

Nama : Atalariq Barra Hadinugraha
NIM : 25/557554/SV/26192
Kelas : PSDA-B2
Dosen Pengampu : Dr. John Doe

**D-IV TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026**

Daftar Isi

A. Tujuan Praktikum	3
B. Dasar Teori	3
1. Abstract Data Type ADT	3
2. Stack	4
3. Queue	4
4. Linked List sebagai Basis Implementasi	4
C. Hasil dan Pembahasan	5
1. Implementasi Node	5
2. Implementasi Stack	5
3. Implementasi Queue	7
4. Analisis Kompleksitas	8
D. Kesimpulan	8
Daftar Pustaka	9

A. Tujuan Praktikum

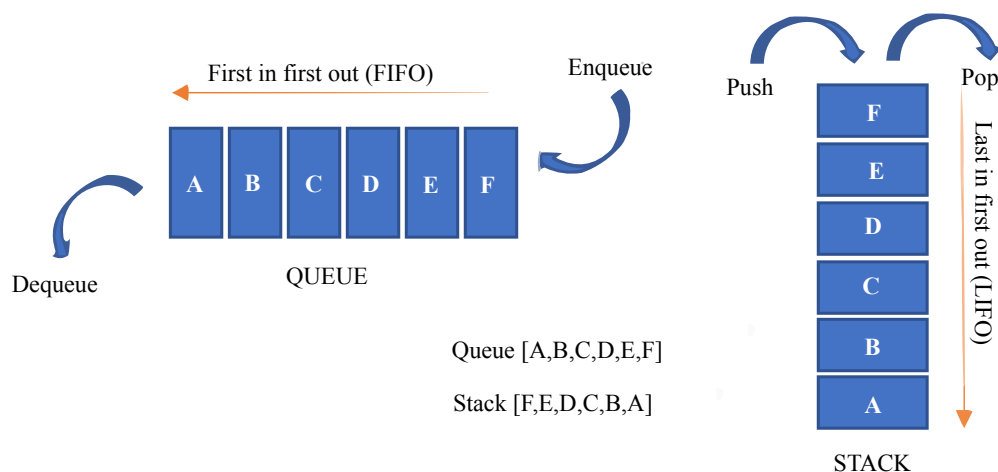
1. Memahami konsep Abstract Data Type (ADT) Stack dan Queue beserta karakteristik urutannya masing-masing.
2. Mengimplementasikan struktur data Stack dan Queue menggunakan Linked List dalam bahasa pemrograman Python.
3. Menganalisis kompleksitas waktu operasi inti Stack dan Queue, meliputi Push, Pop, Enqueue, dan Dequeue.
4. Membedakan karakteristik LIFO dan FIFO serta kasus penggunaan nyata dari masing-masing struktur data.

B. Dasar Teori

1. Abstract Data Type ADT

Abstract Data Type (ADT) adalah model matematis untuk suatu tipe data yang didefinisikan melalui perilakunya dari sudut pandang pengguna, bukan melalui detail implementasinya [1]. ADT memisahkan antara **antarmuka** (operasi apa yang tersedia dan semantiknya) dengan **implementasi** (bagaimana operasi itu dijalankan di memori), sehingga memungkinkan penggantian implementasi tanpa memengaruhi kode yang bergantung padanya [2].

Konsep ini merupakan fondasi dari *information hiding* dan pemrograman modular. Dalam praktiknya, Stack dan Queue adalah dua ADT paling fundamental yang mendefinisikan dua pola akses data yang berlawanan: LIFO dan FIFO [3].



Gambar 1: Queue vs Stack

2. Stack

Stack adalah ADT yang mengikuti disiplin akses *Last In, First Out* (LIFO): elemen yang paling terakhir dimasukkan adalah yang pertama kali dikeluarkan [1]. Operasi-operasi yang didefinisikan oleh ADT Stack adalah:

- Push** Menyisipkan elemen baru ke bagian atas (*top*) Stack.
- Pop** Menghapus dan mengembalikan elemen dari bagian atas Stack. Menghasilkan error jika Stack kosong.
- Peek** Mengakses elemen teratas tanpa menghapusnya.
- isEmpty** Memeriksa apakah Stack tidak memiliki elemen.

Analogi intuitifnya adalah tumpukan piring: piring yang diletakkan paling terakhir harus diambil terlebih dahulu sebelum dapat mengakses piring di bawahnya. Dalam sistem komputer, Stack digunakan secara luas pada mekanisme *call stack* untuk manajemen pemanggilan fungsi rekursif, algoritma *backtracking*, evaluasi ekspresi aritmatika, dan fitur undo/redo pada editor teks [1].

3. Queue

Queue adalah ADT yang mengikuti disiplin akses *First In, First Out* (FIFO): elemen yang pertama kali dimasukkan adalah yang pertama kali dikeluarkan [1]. Operasi-operasinya adalah:

- Enqueue** Menyisipkan elemen baru ke bagian belakang (*rear*) Queue.
- Dequeue** Menghapus dan mengembalikan elemen dari bagian depan (*front*) Queue.
- isEmpty** Memeriksa apakah Queue tidak memiliki elemen.

Berbeda dengan Stack yang hanya membutuhkan satu titik akses (*top*), Queue memerlukan dua pointer: satu ke ujung depan (*front*) untuk operasi Dequeue, dan satu ke ujung belakang (*rear*) untuk operasi Enqueue. Keduanya bergerak maju secara independen seiring operasi yang dilakukan [3]. Queue banyak dipakai pada *task scheduling* sistem operasi, buffer I/O, dan algoritma *Breadth-First Search* (BFS) pada graf.

4. Linked List sebagai Basis Implementasi

Linked List adalah struktur data linear yang terdiri atas simpul-simpul (*node*) yang masing-masing menyimpan data dan sebuah pointer ke simpul berikutnya [1]. Karakteristik ini menjadikannya pilihan alami sebagai basis implementasi Stack dan Queue karena tiga alasan:

1. Alokasi memori bersifat dinamis; ukuran struktur tumbuh dan menyusut mengikuti kebutuhan aktual, tanpa batas kapasitas tetap.

2. Operasi penyisipan dan penghapusan di ujung list berjalan dalam $O(1)$ karena hanya memanipulasi pointer tanpa pergeseran elemen.
3. Tidak ada pemborosan memori akibat pre-alokasi berlebih seperti yang terjadi pada implementasi berbasis array.

Implementasi berbasis array memang menawarkan *cache locality* yang lebih baik, namun memerlukan resize yang berbiaya $O(n)$ saat kapasitas penuh [3]. Untuk kasus umum di mana ukuran data tidak dapat diprediksi, Linked List memberikan garansi $O(1)$ yang konsisten.

C. Hasil dan Pembahasan

1. Implementasi Node

Kedua struktur data Stack dan Queue berbagi komponen dasar yang sama: sebuah kelas Node yang menjadi unit penyusun Linked List.

```
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
```

Setiap Node menyimpan dua atribut: data yang menampung nilai elemen, dan next yang merupakan pointer ke Node berikutnya dalam rantai. Nilai awal next = None menandai bahwa Node baru belum terhubung ke simpul manapun.

2. Implementasi Stack

```
src/stack.py
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5
6 class Stack:
7     def __init__(self):
8         self.top = None
9         self.size = 0
10
11     def push(self, data):
12         node = Node(data)
13         node.next = self.top
14         self.top = node
15         self.size += 1
16
17     def pop(self):
```

```

18         if self.is_empty():
19             raise IndexError("Stack is empty")
20         data = self.top.data
21         self.top = self.top.next
22         self.size -= 1
23         return data
24
25     def peek(self):
26         if self.is_empty():
27             raise IndexError("Stack is empty")
28         return self.top.data
29
30     def is_empty(self):
31         return self.top is None
32

```

Pointer `self.top` bertindak sebagai satu-satunya titik akses ke seluruh struktur. Pada operasi push, Node baru disambungkan ke Node yang sebelumnya berada di *top* (`node.next = self.top`), kemudian `top` diperbarui ke Node baru. Operasi ini bersifat $O(1)$ karena tidak bergantung pada jumlah elemen yang sudah ada.

Pada operasi pop, pointer `top` dimajukan ke Node berikutnya (`self.top = self.top.next`). Node lama tidak lagi direferensikan oleh struktur manapun, sehingga *garbage collector* Python akan secara otomatis membebaskan memorinya. Pemeriksaan `is_empty()` sebelum mengakses `self.top.data` penting untuk mencegah `AttributeError` saat Stack kosong.

Berikut adalah demonstrasi eksekusi Stack:

Eksekusi	Output
<code>s = Stack()</code>	30
<code>s.push(10)</code>	20
<code>s.push(20)</code>	2
<code>s.push(30)</code>	
<code>print(s.pop())</code> # ?	
<code>print(s.peek())</code> # ?	
<code>print(s.size)</code> # ?	

Urutan output 30, 20 mengonfirmasi perilaku LIFO: meskipun 10 di-push pertama, 30 yang di-push terakhir adalah yang pertama dikembalikan. Setelah pop, `top` kini menunjuk ke Node berisi 20, yang dikembalikan oleh peek tanpa mengubah struktur.

3. Implementasi Queue

```
src/queue.py
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5
6
7 class Queue:
8     def __init__(self):
9         self.front = None
10        self.rear = None
11        self.size = 0
12
13    def enqueue(self, data):
14        node = Node(data)
15        if self.rear is not None:
16            self.rear.next = node
17        self.rear = node
18        if self.front is None:
19            self.front = node
20        self.size += 1
21
22    def dequeue(self):
23        if self.is_empty():
24            raise IndexError("Queue is empty")
25        data = self.front.data
26        self.front = self.front.next
27        if self.front is None:
28            self.rear = None
29        self.size -= 1
30        return data
31
32    def is_empty(self):
33        return self.front is None
34
```

Implementasi Queue memerlukan perhatian khusus pada dua kondisi tepi (*edge cases*). Pertama, pada enqueue saat Queue kosong: Node baru harus diassign ke `self.front` sekaligus `self.rear`, karena elemen pertama sekaligus menjadi ujung depan dan belakang. Kedua, pada dequeue saat elemen terakhir dihapus: `self.front` akan menjadi `None`, sehingga `self.rear` juga harus disetel `None` untuk menjaga konsistensi state. Jika kondisi ini diabaikan, `rear` akan menjadi *dangling pointer* yang menunjuk ke Node yang sudah tidak valid.

Eksekusi	Output
<code>q = Queue()</code>	A
<code>q.enqueue("A")</code>	B
<code>q.enqueue("B")</code>	1
<code>q.enqueue("C")</code>	
<code>print(q.dequeue())</code> # ?	
<code>print(q.dequeue())</code> # ?	
<code>print(q.size)</code> # ?	

Urutan output A, B mengonfirmasi perilaku FIFO: A yang masuk pertama adalah yang pertama keluar, terlepas dari urutan elemen di belakangnya.

4. Analisis Kompleksitas

Tabel 1: Kompleksitas Waktu dan Ruang Stack dan Queue Berbasis Linked List

Operasi	Stack	Queue
Push / Enqueue	$O(1)$	$O(1)$
Pop / Dequeue	$O(1)$	$O(1)$
Peek / Front	$O(1)$	$O(1)$
Pencarian (Search)	$O(n)$	$O(n)$
Ruang (Space)	$O(n)$	$O(n)$

Semua operasi inti Stack dan Queue berbasis Linked List berjalan dalam waktu konstan $O(1)$ karena setiap operasi hanya memanipulasi satu atau dua pointer tanpa iterasi [1]. Ini berbeda dengan implementasi berbasis array, di mana operasi push sesekali membutuhkan $O(n)$ saat terjadi resize. Kompleksitas ruang $O(n)$ mencerminkan bahwa total memori yang digunakan sebanding lurus dengan jumlah elemen yang tersimpan, ditambah *overhead* pointer next pada setiap Node.

D. Kesimpulan

Berdasarkan hasil implementasi dan analisis pada praktikum ini, dapat ditarik kesimpulan sebagai berikut:

1. Stack sebagai ADT LIFO berhasil diimplementasikan menggunakan Linked List dengan satu pointer top, di mana operasi push dan pop keduanya berjalan dalam $O(1)$ tanpa overhead alokasi statis.
2. Queue sebagai ADT FIFO memerlukan dua pointer front dan rear untuk mempertahankan efisiensi $O(1)$ pada operasi enqueue dan dequeue; penanganan *edge case* saat Queue kosong atau tinggal satu elemen merupakan aspek kritis implementasi yang benar.
3. Linked List terbukti menjadi basis implementasi yang efisien untuk keduanya, dengan kelebihan utama berupa alokasi memori dinamis yang menghilangkan batasan kapasitas tetap dan operasi manipulasi ujung list yang selalu $O(1)$.

4. Perbedaan mendasar antara Stack dan Queue terletak pada pola akses data: Stack cocok untuk skenario yang memerlukan pembalikan urutan (call stack, backtracking, undo/redo), sedangkan Queue cocok untuk skenario yang memerlukan pemrosesan berurutan (task scheduling, BFS, buffer I/O).

Daftar Pustaka

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, dan C. Stein, *Introduction to Algorithms*, 4th ed. The MIT Press, 2022.
- [2] A. V. Aho, J. E. Hopcroft, dan J. D. Ullman, *Data Structures and Algorithms*. Addison-Wesley, 1983.
- [3] R. Sedgewick dan K. Wayne, *Algorithms*, 4th ed. Addison-Wesley Professional, 2011.
- [4] A. Einstein, “Zur Elektrodynamik bewegter Körper. (German) [On the electrodynamics of moving bodies],” *Annalen der Physik*, vol. 322, no. 10, hlm. 891–921, 1905, doi: <http://dx.doi.org/10.1002/andp.19053221004>.
- [5] D. E. Knuth, “Fundamental Algorithms,” Addison-Wesley, 1973, chap. 1.2.